

Problemas propuestos 1.1

Respuestas:

- 01) Insertar 2 documentos en la colección clientes con '_id' no repetidos.
- ```
db.clientes.insertOne ({_id: 2, nombre: 'Juan Díaz', domicilio: 'Colon 112', provincia: 'Buenos Aires'})
db.clientes.insertOne ({_id: 3, nombre: 'María Perez', domicilio: 'Colon 113', provincia: 'Misiones'})
```
- 02) Intentar insertar otro documento con clave repetida.
- ```
db.clientes.insertOne ({_id: 2, nombre: 'Alex Martínez', domicilio: 'Colon 114', provincia: 'Cordoba'})
```
- 03) Mostrar todos los documentos de la colección libros.
- ```
db.libros.find()
{
 _id: ObjectId('6ac86fe2477c5a7bd790a812'),
 codigo: 3,
 nombre: 'Aprenda PHP',
 autor: 'Mario Molina',
 editoriales: [
 'Planeta'
]
},
{
 _id: ObjectId('6ac86fe2477c5a7bd790a813'),
 codigo: 4,
 nombre: 'Java en 10 minutos',
 autor: 'Barros Sergio',
 editoriales: [
 'Planeta',
 'Siglo XXI'
]
}
```

# Problemas propuestos 1.2

---

- 01) Crear una base de datos llamada blog.
- ```
use blog
```

02) Agregar una coleccion llamada "post" e insertar un documento con una estructura a su eleccion.

```
db.posts.insertOne({titulo: 'Lo desconocido', autor: 'Marcos Perez', genero: ['Vida cotidiana', 'Reflexión'] })
```

03) Mostrar todas las bases de datos actuales.

```
show dbs
```

04) Eliminar la colección "posts"

```
db.posts.drop()  
show collections
```

05) Eliminar la base de datos "blog" y mostrar las bases de datos existentes.

```
db.dropDatabase()  
show dbs
```

Problemas propuestos 1.3

01) Crear la colección 'articulos' en la base de datos 'base1' (eliminar la colección previamente), cargar luego 6 documentos.

```
use base1  
db.articulos.drop()  
use base1  
db.articulos.drop()  
db.articulos.insertOne(  
  {  
    _id: 1,  
    nombre: 'MULTIFUNCION HP DESKJET 2675',  
    rubro: 'impresora',  
    precio: 3000,  
    stock: 20  
  }  
)  
db.articulos.insertOne(  
  {  
    _id: 2,  
    nombre: 'MULTIFUNCION EPSON EXPRESSION XP241',  
    rubro: 'impresora',  
    precio: 3700,  
    stock: 5  
  }  
)
```

```

db.articulos.insertOne(
{
  _id: 3,
  nombre: 'LED 19 PHILIPS',
  rubro: 'monitor',
  precio: 4500,
  stock: 2
}
)
db.articulos.insertOne(
{
  _id: 4,
  nombre: 'LED 22 PHILIPS',
  rubro: 'monitor',
  precio: 5700,
  stock: 4
}
)
db.articulos.insertOne(
{
  _id: 5,
  nombre: 'LED 27 PHILIPS',
  rubro: 'monitor',
  precio: 12000,
  stock: 1
}
)
18
MongoDB Consola
db.articulos.insertOne(
{
  _id: 6,
  nombre: 'LOGITECH M90',
  rubro: 'mouse',
  precio: 300,
  stock: 4
}
)

```

02) Imprimir todos los documentos de la colección 'articulos'.

```
db.articulos.find()
```

03) Imprimir todos los documentos de la colección 'articulos' que no son impresoras.

```
db.articulos.find({ rubro: { $ne: 'impresora' } })
```

- 04) Imprimir todos los artículos que pertenecen al rubro de 'mouse'.
`db.articulos.find({ rubro: 'mouse' })`
- 05) Imprimir todos los artículos con un precio mayor o igual a 5000.
`db.articulos.find({ precio: { $gte: 5000 } })`
- 06) Imprimir todas las impresoras que tienen un precio mayor o igual a 3500.
`db.articulos.find({ rubro: 'impresora', precio: { $gte: 3500 } })`
- 07) Imprimir todos los artículos cuyo stock se encuentra comprendido entre 0 y 4.
`db.articulos.find({ stock: { $gte: 0, $lte: 4 } })`

Problemas propuestos 1.4

- 01) Crear la colección 'articulos' en la base de datos 'base1' (eliminar la colección previamente), cargar luego 6 documentos: Utilizar la colección del problema propuesto anterior (1.3).
Ya creada en 1.3
- 02) Imprimir todos los documentos de la colección 'articulos'.
`db.articulos.find()`
- 03) Borrar los documentos de la colección 'articulos' cuyo rubro son impresoras, utilizar las dos sintaxis que permite MongoDB.
`db.articulos.deleteMany({ rubro: 'impresora' })`
`db.articulos.deleteMany({ rubro: { $eq: 'impresora' } })`
- 04) Borrar todos los artículos que tienen un _id mayor o igual a 5.
`db.articulos.deleteMany({ _id: { $gte: 5 } })`

Problemas propuestos 1.5

- 01) Crear la colección 'articulos' en la base de datos 'base1' (eliminar la colección previamente), cargar luego 6 documentos: Utilizar la colección del problema propuesto anterior (1.3).
Ya creada
- 02) Imprimir todos los documentos de la colección 'articulos'.
`db.articulos.find()`

03) Modificar el precio del mouse 'LOGITECH M90'.

```
db.articulos.updateOne(  
  { nombre: 'LOGITECH M90' },  
  { $set: { precio: 350 } }  
)
```

04) Fijar el stock en 0 del artículo cuyo _id es 6.

```
db.articulos.updateOne(  
  { _id: 6 },  
  { $set: { stock: 0 } }  
)
```

05) Agregar el campo proveedores con el array ['Martinez','Gutierrez'] para el artículo cuyo _id es 6.

```
db.articulos.updateOne(  
  { _id: 6 },  
  { $set: { proveedores: ['Martinez', 'Gutierrez'] } }  
)
```

06) Eliminar el campo proveedores para el artículo cuyo _id es 6.

```
db.articulos.updateOne(  
  { _id: 6 },  
  { $unset: { proveedores: '' } }  
)
```

Problemas propuestos 1.6

01) Crear la colección 'articulos' en la base de datos 'base1' (eliminar la colección previamente), cargar luego 6 documentos: Utilizar la colección del problema propuesto anterior (1.3).

Ya creada.

02) Imprimir todos los documentos de la colección 'articulos'.

```
db.articulos.find()
```

03) Fijar el stock en cero para todos los artículos del rubro monitor.

```
db.articulos.updateMany(  
  { rubro: 'monitor' },  
  { $set: { stock: 0 } }  
)
```

- 04) Agregar un campo llamado 'pedir' con el valor true para todos los artículos que tienen el campo stock en 0.

```
db.articulos.updateMany(  
  { stock: 0 },  
  { $set: { pedir: true } }  
)
```

- 05) Eliminar el campo 'pedir' de todos los documentos.

```
db.articulos.updateMany(  
  {},  
  { $unset: { pedir: " " } }  
)
```

Problemas propuestos 1.7

- 01) Crear la colección 'medicamentos' en la base de datos 'base1' (eliminar la colección previamente), cargar luego 6 documentos:

```
use base1
```

```
db.medicamentos.drop()
```

```
db.medicamentos.insertOne({ _id: 1, nombre: 'Sertal', laboratorio: 'Roche', precio: 5.2,  
cantidad: 100 })
```

```
db.medicamentos.insertOne({ _id: 2, nombre: 'Buscapina', laboratorio: 'Roche', precio:  
4.10, cantidad: 200 })
```

```
db.medicamentos.insertOne({ _id: 3, nombre: 'Amoxidal 500', laboratorio: 'Bayer',  
precio: 15.60, cantidad: 100 })
```

```
db.medicamentos.insertOne({ _id: 4, nombre: 'Paracetamol 500', laboratorio: 'Bago',  
precio: 1.90, cantidad: 200 })
```

```
db.medicamentos.insertOne({ _id: 5, nombre: 'Bayaspirina', laboratorio: 'Bayer',  
precio: 2.10, cantidad: 150 })
```

```
db.medicamentos.insertOne({ _id: 6, nombre: 'Amoxidal jarabe', laboratorio: 'Bayer',  
precio: 5.10, cantidad: 50 })
```

- 02) Imprimir todos los documentos.

```
db.medicamentos.find()
```

- 03) Recupere los medicamentos cuyo laboratorio sea 'Roche' y cuyo precio sea menor a 5.

```
db.medicamentos.find({ laboratorio: 'Roche', precio: { $lt: 5 } })
```

- 04) Recupere los medicamentos cuyo laboratorio sea 'Roche' o cuyo precio sea menor a 5.

```
db.medicamentos.find({ $or: [ { laboratorio: 'Roche' }, { precio: { $lt: 5 } } ] })
```

- 05) Muestre todos los medicamentos cuyo laboratorio NO sea "Bayer".

```
db.medicamentos.find({ laboratorio: { $ne: 'Bayer' } })
```

- 06) Muestre todos los medicamentos cuyo laboratorio sea "Bayer" y cuya cantidad NO sea=100.

```
db.medicamentos.find({ laboratorio: 'Bayer', cantidad: { $ne: 100 } })
```

- 07) Elimine todos los documentos de la colección medicamentos cuyo laboratorio sea igual a "Bayer" y su precio sea mayor a 10.

```
db.medicamentos.deleteMany({ laboratorio: 'Bayer', precio: { $gt: 10 } })
```

- 08) Cambie la cantidad por 200, a todos los medicamentos de "Roche" cuyo precio sea mayor a 5.

```
db.medicamentos.updateMany(  
  { laboratorio: 'Roche', precio: { $gt: 5 } },  
  { $set: { cantidad: 200 } }  
)
```

- 09) Borre los medicamentos cuyo laboratorio sea "Bayer" o cuyo precio sea menor a 3.

```
db.medicamentos.deleteMany({ $or: [ { laboratorio: 'Bayer' }, { precio: { $lt: 3 } } ] })
```

Problemas propuestos 1.8

- 01) Crear la colección 'articulos' en la base de datos 'base1' (eliminar la colección previamente), cargar luego 6 documentos.

```
use base1
```

```
db.articulos.drop()
```

```
db.articulos.insertOne({ _id: 1, nombre: 'MULTIFUNCION HP DESKJET 2675', rubro: 'impresora', precio: 3000, stock: 20 })
```

```
db.articulos.insertOne({ _id: 2, nombre: 'MULTIFUNCION EPSON EXPRESSION XP241', rubro: 'impresora', precio: 3700, stock: 5 })
```

```
db.articulos.insertOne({ _id: 3, nombre: 'LED 19 PHILIPS', rubro: 'monitor', precio: 4500, stock: 2 })
```

```
db.articulos.insertOne({ _id: 4, nombre: 'LED 22 PHILIPS', rubro: 'monitor', precio: 5700, stock: 4 })
```

```
db.articulos.insertOne({ _id: 5, nombre: 'LED 27 PHILIPS', rubro: 'monitor', precio: 12000, stock: 1 })
```

```
db.articulos.insertOne({ _id: 6, nombre: 'LOGITECH M90', rubro: 'mouse', precio: 300, stock: 4 })
```

- 02) Imprimir todos los documentos, mostrar solo los campos _id y nombre.

```
db.articulos.find({}, { _id: 1, nombre: 1 })
```

03) Imprimir todos los documentos que son impresoras, mostrar solo los campos nombre y precio.

```
db.articulos.find({ rubro: 'impresora' }, { _id: 0, nombre: 1, precio: 1 })
```

04) Imprimir todas las impresoras que tienen un precio mayor o igual a 3500. Solo mostrar los campos _id, nombre, precio y stock.

```
db.articulos.find(  
  { rubro: 'impresora', precio: { $gte: 3500 } },  
  { _id: 1, nombre: 1, precio: 1, stock: 1 }  
)
```

05) Imprimir todos los documentos que son monitor, mostrar solo los campos nombre y precio ordenados de menor a mayor.

```
db.articulos.find(  
  { rubro: 'monitor' },  
  { _id: 0, nombre: 1, precio: 1 }  
) .sort({ precio: 1 })
```